

«Магическая пиллюля»: модель ответит точнее, если попросить её экспертом и усложнить промпт.



Усложнять промпт ради точности: «ты — эксперт-юрист, рассуждай по шагам»



Выбрать архитектуру под задачу: контекст, RAG или инструмент

Откуда на самом деле берётся надёжность AI-системы — разбираем всю лекцию.

Архитектуры AI-систем: агенты, RAG, API

03

Какую архитектуру выбрать под задачу —
и когда правильный ответ «не ИИ»



«магической пилюли» не существует

Маршрут лекции — шесть разделов.

Одна несущая линия: выбор архитектуры под задачу — и когда правильный ответ «не ИИ».

0



Открытие

постановка задачи: откуда берётся надёжность

1



Промпт и его границы

что умеет один вызов и где его потолок

2



RAG

внешнее знание в контекст — и где оно тихо ломается

3



Дообучение

менять веса под поведение, не под знание

4



Агенты

цикл, экипировка, память, безопасность — и провалы

5



Фреймворк

лестница + чек-лист: как выбрать быстро и обоснованно

Из Лекции 2 берём готовыми два понятия.

Не переобъясняем — просто вспоминаем, чтобы дальше на них опереться. Всё остальное надстроим по ходу лекции.



Одиночный вызов (single-shot)

Один проход модели: дал промпт → получил ответ. Без памяти между вызовами, без обращений наружу.

Модель знает только то, что в промпте и в весах



Семантический поиск на эмбедингах

Текст → вектор смысла; близкие по смыслу фрагменты — близкие векторы. Поиск «по смыслу», а не по точному слову.

На этом стоит RAG (Раздел 2)

Оба понятия — фундамент из Лекции 2. Всё, что мы надстроим сегодня (RAG, инструменты, дообучение, агенты), опирается на них.

У меня есть задача и доступ к LLM. Какую архитектуру выбрать — и когда правильный ответ «не ИИ»?

6

Мульти-агент

несколько координируемых агентов

сложнее ↑

5

Агент

цикл: план → действие → проверка → повтор (plan → act → check → iterate)

4

Сценарий (workflow)

предопределённые пути

3

RAG / контекст-инжиниринг

поиск-дополненная генерация

2

Один вызов LLM

промт; + CoT (по шагам), + примеры в промте (few-shot)

1

Обычный код (без ИИ)

точка отсчёта

проще ↓



Подниматься на следующую ступень — только при требовании задачи, которого текущая не закрывает.

РАЗДЕЛ 1

Промпт и его границы

Лестницу мы увидели целиком — теперь снизу: что умеет один вызов и где его потолок, прежде чем что-либо усложнять.

один точный рез · 3 разбора · 1 провал

Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Дообучение

Раздел 4
Агенты

Раздел 5
Фреймворк

Что такое точная постановка задачи: спецификация и контракт.

Точная постановка — это описание нужного результата так, чтобы его можно было проверить: спецификация, детерминированная программа, контракт интерфейса. Отсюда стартует раздел.



Точная спецификация / ТЗ

Пред- и постусловия, инварианты, критерии приёмки (Z-нотация, TLA+, Design by Contract). Результат детерминирован и проверяем.



Императив vs декларатив

«Как сделать» (алгоритм по шагам) против «что получить» (SQL/Prolog описывает результат, движок решает как).



Контракт интерфейса

Точное соглашение вход/выход (сигнатуры типов, OpenAPI, Protobuf). Единственный корректный смысл и способ его проверить.

ЧТО ОСТАВИТЬ

Дисциплину точной постановки: точный промт — то же ТЗ на естественном языке. Для детерминированного и верифицируемого (арифметика, валидация по схеме, маршрутизация по правилам) — классический код, а не промт.

Мост: промт — постановка задачи вероятностной системе на естественном языке; отсюда и сила (не нужно специфицировать неподъёмное), и граница (нет единственного смысла, нет детерминизма).

По умолчанию — один вызов с хорошим промптом.

Модель знает только то, что в промпте (плюс то, что было в весах на обучении). Больше ей взять неоткуда.



Один вызов LLM с хорошим промптом

- низкая стоимость (один проход)
- низкая задержка (нет лишних обращений)
- высокая предсказуемость (нет петель, нет поиска, который тихо деградирует)

«НА ВСЯКИЙ СЛУЧАЙ» — ЭСКАЛАЦИЯ



Не усложняй архитектуру без причины, выраженной в требованиях задачи^[1]. Это распределение бремени доказательства, а не примитивизм.

[1] Anthropic — Building Effective Agents («найди простейшее»)

«Роль» — это два разных механизма. Не путайте их.

Роль-персона («ты — юрист») настраивает тон. Протокольная роль system/user/assistant — это разметка «кто говорит» в потоке токенов. Второе постоянно принимают за первое.

{ } Chat-шаблон: список ролей → плоский поток токенов

```
<|im_start|>system правила поведения <|im_end|>  
<|im_start|>user вопрос <|im_end|>
```

Спецтокены разметки (ChatML, Llama 4) собираются шаблоном — его можно прочитать и подменить.

Приоритет system — склонность, не граница

GPT-4o слушается приоритета ~**63,8%** — всё равно ≠ 100%: это склонность.



STI / подделка роли (role spoofing)

Строка `<|im_start|>assistant` из внешнего текста → ChatInject ASR
5,18% → 32,05% (Llama-4 до 88,3%).

Что делать: не проектируйте защиту в расчёте «система важнее пользователя всегда». Экранируйте спецтокены во **ВХОДЯЩЕМ** внешнем контенте и проверяйте chat-шаблон у локальных моделей — чужой шаблон тихо ломает приоритет.



Структура промпта: разделяй инструкцию, контекст, данные.

Плоский промпт заставляет модель угадывать, где кончается инструкция и начинаются данные. Явная граница снимает эту неоднозначность.



Инструкция

что сделать



Контекст

на основе чего



Данные

с чем именно работать

Разделители (delimiters)

- XML-теги: `<инструкция>...</инструкция>`
- Markdown-заголовки: `## Задача · ## Данные`
- Тройные кавычки / бэктики вокруг данных

Тот же принцип, что структурированный вывод (Раздел 4): там схема задаёт структуру ВЫХОДА, здесь разделители — структуру ВХОДА. Ясная структура вместо «выведи форму по смыслу».

Модель, которой показали, где кончается инструкция и начинаются данные, реже принимает фрагмент данных за новую команду — та же путаница лежит в основе инъекции в промпт (prompt injection) (Раздел 4).

Формат промпта: вход терпим, выход — с ценой.

Терпимость к формату ВХОДА и цена принуждения формата ВЫХОДА — две разные проблемы. Их постоянно смешивают.



Markdown

простой запрос, инструкции

GPT/Gemini · вход терпим



XML-теги

длинный контекст, много блоков

Claude · разграничить блоки



JSON + схема

извлечение, классификация, код

строгое декодирование → ~100%



JSON на рассуждении

не применять к «подумай и реши»

налог на рассуждение



JAKE-CLARK.TUMBLR

Не принуждай структуру ВЫХОДА на рассуждающих задачах: GSM8K падает 76,6% → 49,3% (та же модель, только требование JSON). Митигация — двухходовка «сначала свободно рассуждать, потом переформатировать» (+6,8 п.п.).

Chain-of-thought помогает — но его нельзя аудировать.

Без CoT

«Было 23 яблока, 7 испортились, докупили 2 ящика по 6.
Сколько хороших?»

→ правдоподобное, но неверное число

С CoT («решай по шагам»)

$$23 - 7 = 16^{[1]} \quad \cdot \quad 2 \times 6 = 12$$

$$16 + 12 = 28$$

→ верно

Технически это всё ещё один вызов — CoT инструмент под класс задач (цепочка шагов), не глобальный тумблер.

Но проговорённое рассуждение не обязано отражать реальную причину ответа^[2] (низкая верность объяснения):



Проверяй результат, а не самообъяснение модели.

Сверяй факты с внешним источником — правдоподобный текст рассуждения ничего не подтверждает.

Рассуждение вслух — не журнал аудита.

Faithfulness (верность объяснения) — насколько проговорённая цепочка отражает реальную причину ответа. Измерили напрямую — и она низкая.

Эксперимент Anthropic (апр. 2025)



Задачу дают дважды: без подсказки и с подсказкой в промпте, которая меняет ответ («профессор считает, что C»)



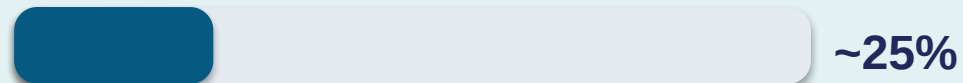
Смотрят: когда ответ изменился под подсказку — упомянула ли модель её в рассуждении?



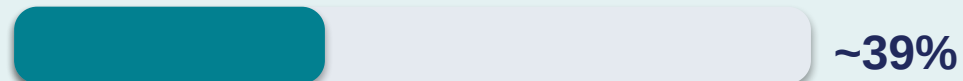
Часто модель меняет ответ, но строит ДРУГУЮ, выдуманную аргументацию, не называя реальную причину

Как часто модель признаёт подсказку

Claude 3.7 Sonnet



DeepSeek R1

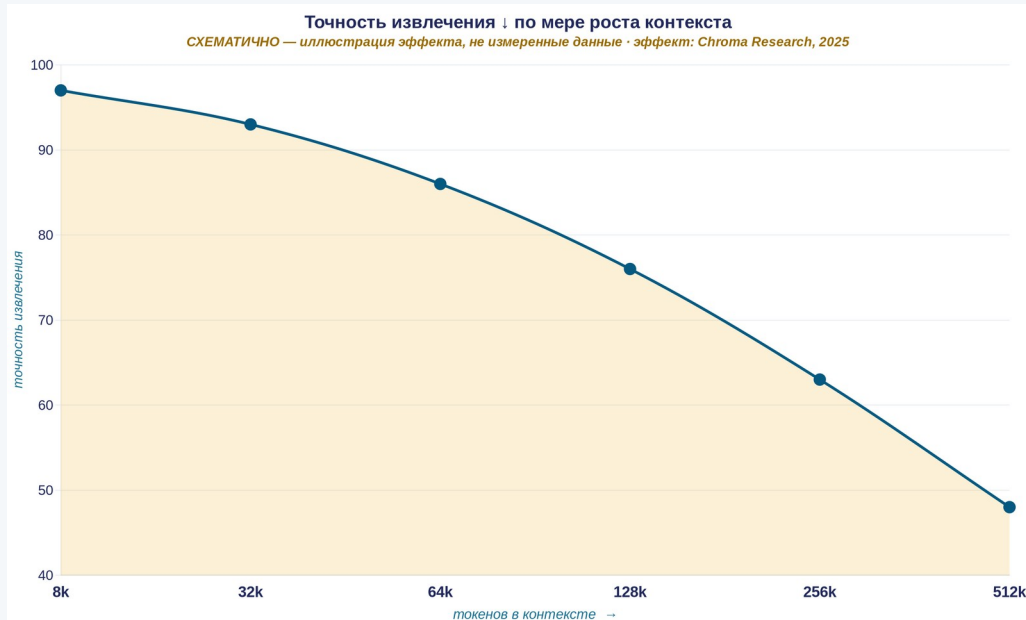


И хуже — на трудных задачах (GPQA ниже MMLU): где аудит нужнее всего, объяснению верить можно меньше всего.

Что делать: человек-валидатор проверяет РЕЗУЛЬТАТ против независимого источника (база, документ, расчёт, эксперт), а не приложенную «цепочку рассуждений». Контроль на самообъяснении модели — не контроль.

Контекст-инжиниринг: минимум высокосигнального.

Промпт-инжиниринг — одна инструкция. Контекст-инжиниринг — курирование всего набора токенов, видимых модели на инференсе.



Когда НЕ RAG (точка 1)

малый стабильный корпус, влезает в окно →
полный контекст + кэширование префикса, а
не RAG-инфраструктура



RAG здесь добавил бы хрупкость
без выигрыша

context rot = тот же «lost in the middle» из L2 [1,2] — новый термин, не новая сущность

«Найти наименьший набор высокосигнальных токенов, максимизирующий вероятность желаемого исхода» — это инженерное требование, не эстетика [3].

Шпаргалка: как строить промпт.

Минимум, ниже которого промпт систематически недорабатывает. Приложите к любому промпту перед первым запуском.

1 Роль

если нужен тон/регистр — и НЕ как обещание точности

2 Задача

конкретное проверяемое действие, не расплывчатое пожелание

3 Контекст

минимально необходимое, не «всё, что могло бы пригодиться»

4 Формат вывода

указан явно, если ответ машинно-обрабатываем

5 Разделители

если содержимого больше одного вида (инструкция/данные)

6 Примеры (few-shot)

только если формат неочевиден из одной инструкции

7 CoT

только если задача требует многошагового рассуждения

8 Длина

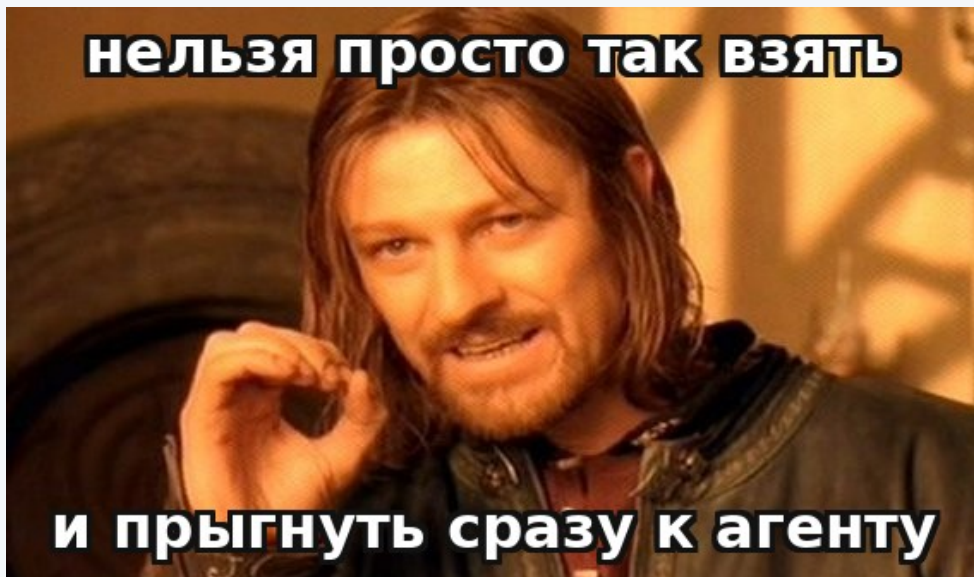
не длиннее необходимого — лишние токены «тонут» в контексте

Компактная форма всего раздела: роль ≠ точность, структура помогает разделить входы, CoT — точечный инструмент, контекст — минимальный. Для крупных архитектурных решений (RAG / FT / агент) — восьмишаговый чек-лист Раздела 5.

Как это выглядит на реальных задачах — типовые классы уровня промпта.

ТИПОВАЯ ЗАДАЧА · 1 · пограничный случай

Ассистент с инструментами: поднимайся по мини-лестнице.



Постановка

Пользователь просит не «текст», а «сделай»: узнать во внешней системе и/или совершить действие.



One-shot → tool-use → агент

Знает ответ, действие не нужно → один вызов. Доступ нужен, шаги известны → вызов инструмента в коде. Шаги неизвестны, есть чем проверить прогресс → цикл ReAct.

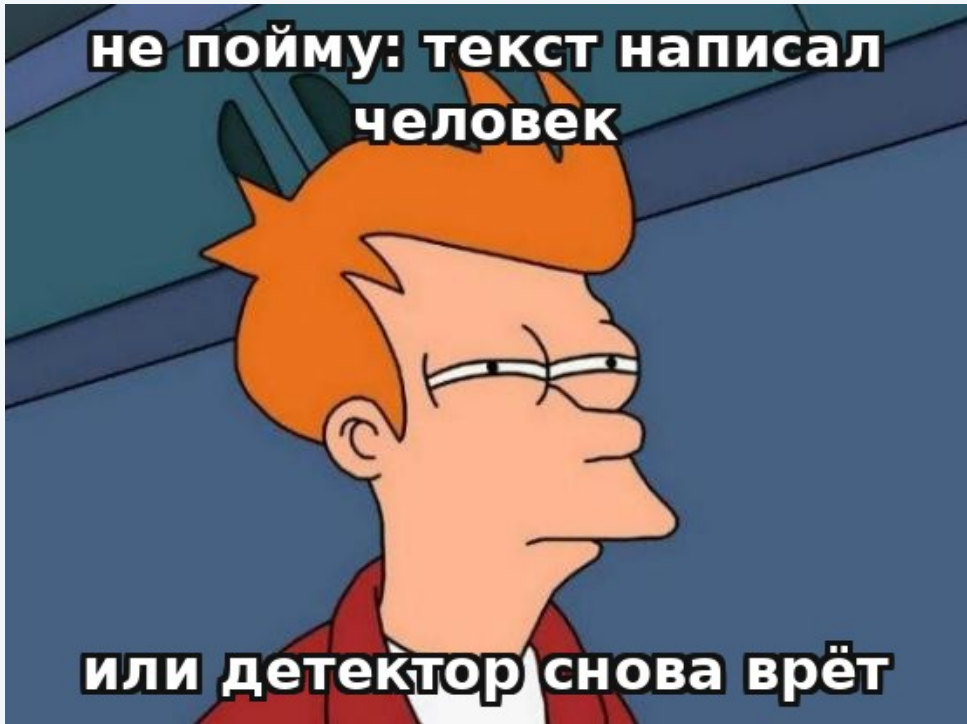
Параметры: гранулярность инструментов, tool_choice, лимит итераций и бюджет токенов, строгие схемы входа.

Валидация: проверять аргументы вызова ДО обращения к системе; ответ опирается на результаты инструментов, не на память.

Типовой провал: прыжок сразу к агенту там, где хватило бы одного вызова, — та же петля на \$4 200 (Раздел 4), где предсказуемую синхронизацию отдали агенту вместо retry-скрипта.

Подниматься по мини-лестнице только под требование задачи — не потому, что «агент звучит мощнее».

Текст в заданном тоне: few-shot по образцам.



Постановка

Нужен текст в определённом голосе, стиле, регистре: бренд-письмо, ответ поддержки, черновик «как человек».



Persona-prompt + few-shot style

Роль-персона (тон, не факты) + короткий style-guide + 3–5 образцов манеры в примерах → генерация в том же стиле.

Параметры: формальность, эмоциональная окраска, персона, число и качество образцов, явные запреты (без канцелярита / без эмодзи).

Валидация: стиль слабо формализуем → LLM-судья на тон + проверка маркеров (длина фраз, лексика) + выборочная вычитка человеком.

Провал суждения: детекторам нельзя верить как доказательству — часть даёт точность ниже 80%, тексты не-нейтивов флагаются как ИИ до +30% чаще.

Как правильно задать JSON-спеку в промпте.

«Попросить JSON» ≠ «задать контракт». Есть два уровня контроля с разными гарантиями, и три способа описать схему.

Два уровня контроля

Промпт-уровень («верни JSON по схеме») — **≈80% валидности**, ничего не гарантирует жёстко.

Декодер (строгие структурированные выходы / строгое декодирование, structured outputs) — схема → грамматика, недопустимые токены маскируются → **~100% соответствия схеме**.

Три способа описать форму

- **Пример-объект**
понятен модели; НЕ выражает типы и enum — плоские структуры, прототип
- **JSON Schema**
типы, enum, required, вложенность; в API напрямую, но многословен + деградирует на сложной
- **Тип в коде (TS / Pydantic)**
компактно + типобезопасно; нужен конвертер в JSON Schema

ПЛОХО

«Извлеки данные и верни JSON» — нет схемы, имён полей, типов, enum; модель обернёт в markdown и придумает поля.

ХОРОШО

```
"rating": number    // 1-5, дробное; null если нет
"sentiment": string // one of [positive,neutral,negative]
"pros": string[]    // [] если нет
+ few-shot «вход → ожидаемый JSON», схема В КОНЦЕ,
«Верни ТОЛЬКО JSON, без markdown».
```

Налог на рассуждение

не рассуждать в JSON: «сначала свободно подумай → потом переформатируй» отдельным шагом

Сложная схема ↓

точность извлечения ~87% → 70% → 56%:
дробить, вложенность мелкой

Валидный JSON ≠ правильные данные: грамматика гарантирует форму, не истинность значений — каждый JSON всё равно проверяется в коде. Строгая схема помогает на извлечении / классификации / вызове функций (function calling); на «подумай и реши» — налог на рассуждение.

РАЗДЕЛ 2

RAG: поиск-дополненная генерация

Извлечь релевантное → положить в контекст → ответить с опорой на источник

внешнее знание · 5 архетипов · 2 провала

Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Дообучение

Раздел 4
Агенты

Раздел 5
Фреймворк

2

Классический поиск: инвертированный индекс, BM25 и где он работает.

Буква *R* в RAG — *retrieval*, поиск: инвертированный индекс, булев отбор, ранжирование BM25. Их устройство определяет, какой запрос каким поиском брать.



Инвертированный индекс

Для каждого СЛОВА — список документов, где оно есть (машинный каталог). Хребет Lucene, Elasticsearch, PostgreSQL FTS.



Булев отбор

Запрос как логика («ошибка AND аутентификация NOT tomcat»): точный, предсказуемый, объяснимый отбор.



TF-IDF → BM25 (~1994)

Ранжирование по важности слова (реже в коллекции — сильнее сигнал). BM25 (Okapi) — дешёвая линия, которую «умные» системы не всегда обгоняют.

ГДЕ КАКОЙ ПОИСК РАБОТАЕТ

Лексический (BM25)

точные термины, коды ошибок, ID, имена, номера — «ORA-00942» найдёт буквально

Семантический (эмбединги)

перифраз и синонимы, смысл — «сломанный вход» ≈ «authentication failure»

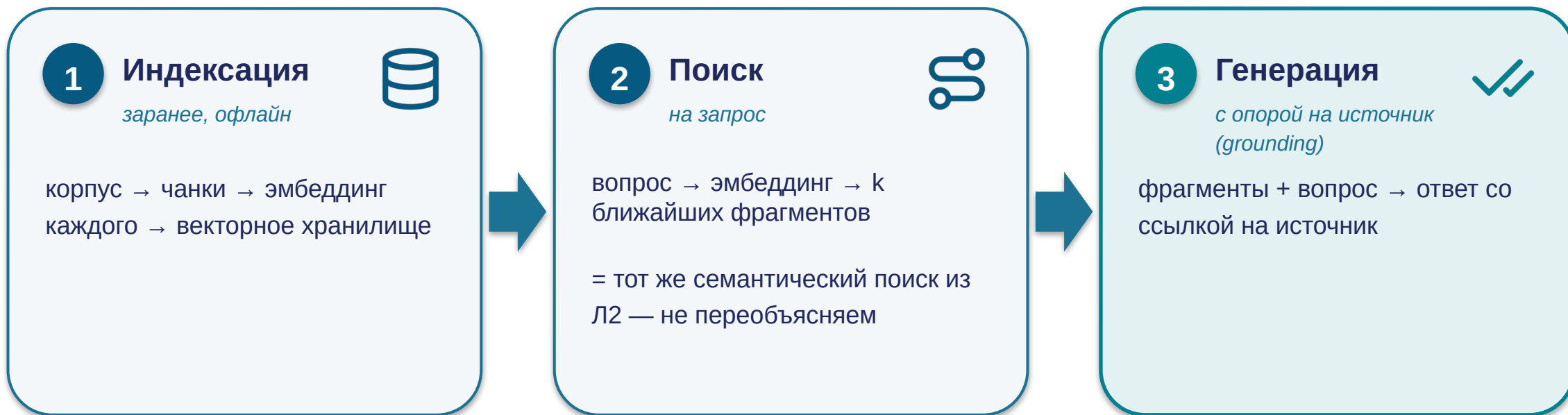
Гибрид (BM25 + векторы)

нужны оба сразу: точная лексика И смысл — дефолт сильного RAG-2026

Ключ: «семантический поиск построен на BM25» — НЕ так. Лексика (BM25, ~1994) и плотные векторы (DPR, 2018–2020) — две независимые ветки; их СЛИВАЮТ (fuse), а не наследуют. RAG = объединение двух веток, а не надстройка над одной.

Принцип RAG — три шага.

RAG = индексация → поиск (семантический поиск из Л2) → генерация с опорой; «не знаю» — корректный ответ.



«Не знаю» / «см. источник X» — корректный ответ RAG-системы^[1]. Правдоподобный ответ при нерелевантном поиске — это дефект, а не «хоть что-то».

RAG-2026: agentic (агентный) RAG по умолчанию · гибридный поиск (BM25 + плотные векторы) · реранкер · каскад промахов 5,7% → 1,9% (Contextual Retrieval).

[1] [Lewis et al. 2020 — Retrieval-Augmented Generation \(NeurIPS\)](#)

«Гибрид» — три разных смысла на общей базе.

Разреженный (sparse) = классический лексический поиск: BM25 / TF-IDF (и learned-sparse SPLADE / ELSER) на инвертированном индексе; измерение = слово. «Разреженный вектор» — это лишь запись пословного поиска (во весь словарь, почти все нули); это НЕ поиск смысла по вектору.

Плотный (dense) = семантический эмбединг (≠ разреженный): 384–1024 значения float, ANN-индекс, не интерпретируем, ловит смысл и перифразу.

(а) лексика + смысл

разреженный (лексика, BM25) + плотный (смысл, эмбединг), слиты по рангам (RRF) — базовый смысл слова «гибрид»

(б) смысл + фильтры

семантика + строгий отбор по метаданным (юрисдикция, дата) — ортогональный рычаг

(в) две лексические ветви

BM25 + SPLADE/ELSER — обе разреженные (классика + нейро-взвешенная), без dense-векторов смысла



Слияние RRF

$\text{score} = \sum 1 / (k + \text{ранг})$, $k \approx 60$ — объединяет два списка рангов без общей шкалы очков.



Реранкер (cross-encoder)

bi-encoder кодирует отдельно (быстро); cross-encoder гоняет пару вместе (точнее, дорого) — только по top-50...100.

WANDS · NDCG

0,7497

vs BM25 0,6983 / вектор 0,6953 → ~7,4% (скромно)^[1]

Финтекст+таблицы · Recall@5

0,816

vs dense-only 0,587 (+0,229)

Anthropic · промахи

5,7 → 1,9%

+контекст+BM25 → 2,9%; +реранк → 1,9% (–67%)^[2]

Выигрыш гибрида зависит от корпуса: крупные приросты — там, где лексика ИЛИ смысл ломается по отдельности (жаргон, таблицы, перекрёстные ссылки). На чистой прозе разрыв мал — сначала измерь на своих запросах, потом усложняй.

Где живут векторы: две категории движков.

Elastic и OpenSearch — тоже полноценные векторные хранилища (dense_vector + HNSW / k-NN плагин): их честнее ставить под осью «поисковый движок, который умеет вектора» vs vector-native (Qdrant / Milvus / Weaviate). Обе — векторные БД; различает происхождение. Потолки движутся — сверять в день лекции.

Движок	Брать когда	Ориентир потолка
 pgvector	у вас уже есть Postgres, нужна одна система и транзакции	~50M векторов; за ~50–100M деградирует ^[1]
 Elastic / OpenSearch	search-engine-first: уже эксплуатируете кластер, нужен нативный BM25+kNN гибрид	млн–десятки млн; ELSER без GPU
 Qdrant	vector-native без большой платформы, sparse + multi-vector	крупный узел + кластер
 Weaviate	vector-native со встроенным гибридом (vector+BM25+фильтры)	средне-крупный
 Milvus	vector-native: 100M+ / миллиарды, тяжёлое горизонтальное масштабирование	миллиарды (нужна эксплуатация) ^[2]
 FAISS · Chroma · LanceDB	FAISS — библиотека (хранение на вас); Chroma/LanceDB — прототип/edge	от прототипа до embedded

Обязки дополняют, не конкурируют

LlamaIndex — ingestion + retrieval по грязным документам; LangGraph — оркестрация (checkpointing, human-in-the-loop).

Для простого RAG-эндпоинта (retrieve → generate) фреймворк не нужен: несколько сотен строк своей склейки против частой смены версий. Обязку добавляют под агентную сложность, а не по умолчанию.

[1] [firecrawl — Best Vector Databases 2026 \(потолки движков\)](#) · [2] [TigerData — pgvector vs Qdrant \(вендор-бенчмарк, читать скептически\)](#)

Что выбрать под свою ситуацию.

Выбор в открытом поле — под свой масштаб и требования. Три яруса снизу вверх; поднимаешься на следующий, только когда предыдущего не хватает под конкретную нагрузку.

**«срочно ставим
векторную БД!»**

**у тебя 3 млн чанков —
хватит Postgres**



Хватает BM25

1

общий словарь, код/логи/ID, курированные базы; переиндексация дёшева и наблюдаема — векторов и вектор-хранилища не нужно

Хватает поискового движка с векторами

2

нужен смысл поверх лексики на масштабе млн–десятки млн; Elastic/OpenSearch дают гибрид и ELSER без GPU-сервиса эмбедингов^[2]

Нужна выделенная векторная БД

3

100M+ / multi-vector / latency-SLA горячего пути / развязать хранилище от эксплуатации кластера логов

Огромная доля RAG-систем никогда не превышает несколько миллионов чанков — на этом масштабе pgvector или гибрид Elastic/OpenSearch скучный правильный ответ^[1], а выделенная векторная БД часто преждевременная оптимизация.

[1] [Pureinsights — From Vector Hype to Hybrid Reality \(нужна ли вектор-БД\)](#) · [2] [bigdataboutique — Elastic/OpenSearch vector search \(RRF-гибрид нативно\)](#)

Чанкирование: как режешь — то и можно найти.

Из трёх шагов RAG чанкирование недооценивают чаще всего, а рычаг высокий. Стратегии — от простой к сложной; усложнять только под требование.



Fixed-size

резать каждые N токенов — грубо, но сильная базовая линия



Recursive / character

по иерархии разделителей (абзац → предложение) — прагматичный дефолт^[1]



Sentence-window

эмбеддить предложение, возвращать окно соседних для контекста



Semantic

граница по смене темы — интуитивно, но часто не окупается



Parent-document

маленькие чанки для точности, больший родитель — для генерации



Late chunking

эмбеддить весь документ, потом пулить — чанк контекст-осведомлён



Contextual Retrieval

LLM дописывает 50–100 токенов документ-уровневого контекста к каждому чанку до эмбединга и BM25^[2] (см. каскад промахов 5,7% → 1,9%).

Ни у размера, ни у стратегии нет правильного значения по умолчанию — их подбирают под корпус. Подбор без метрики качества retrieval — вслепую.

Не карго-культу semantic chunking.

Исследования эффекта чанкирования открыто расходятся — и это само по себе урок: «умный» chunking доменно-специфичен, простой recursive fixed-size часто сильнее и дешевле.



recursive-512 vs semantic

69% vs 54%

Feb-2026, 7 стратегий на 50 статьях: recursive 512-token №1; semantic дал ~43-токенные фрагменты^[1]

клиника (узкий домен)

87% vs 13%

MDPI ноя-2025: adaptive/topic-boundary vs fixed-size ($p=0,001$) — реальный, но узкодоменный

NAACL 2025 Findings

≈ или хуже

«издержки не оправданы стабильным приростом»: fixed 200-word на уровне semantic или лучше

Один приём и выигрывает 74 пункта (клиника), и проигрывает 15 (общий корпус): корпуса, метрики и размер чанка разные. Дефолт — recursive fixed-size; усложнять только под разрыв, измеренный на СВОИХ данных.

[1] firecrawl — chunking benchmark (recursive-512 69% vs semantic 54%) · [2] firecrawl — chunking eval (recall@k в изоляции может врать) · Recall@k в изоляции может врать (91,9% recall при 54% ответов) — мерить и end-to-end ассурасу; следующий слайд — как это ломается тихо.

Тихий провал чанкирования: голый ряд цифр.

Worked example — 300 PDF-руководств с таблицами спецификаций. Вопрос «какой момент затяжки для модели X-500?» — ответ в ячейке таблицы. Наивный сплиттер режет её по байту.

Наивно: RecursiveCharacter, 512 токенов

Парсер расплющил таблицу в поток; сплиттер режет по байту на границе 512:

чанк: «12 / 480 / 8.5 / 34»

Заголовки «Модель / Напряжение / Ток / Момент» ушли в другой чанк. Система НЕ падает: эмбединг считается, ретрив «находит», ответ генерируется — и он неверный.

Правильно: table-aware + повтор заголовков

Таблицу — отдельным чанком; если больше лимита — по строкам, повторяя заголовки:

«Модель X-500 | Напряжение 480 |
Ток 8.5 | Момент 34»

Каждая строка самодостаточна; + parent-document для текста, + метаданные {модель, тип, версия}. Основной выигрыш дало не «умную» нарезку, а уважение структуры.^[1]



Анафора / сломанные ссылки

«оно», «этот город», «данная политика» теряют антецедент при разрезе → чанк эмбеждится неоднозначно (лечить parent-doc / контекст-дописыванием).



Смена эмбеддера = пере-нарезка

размер чанка привязан к окну модели; сменил эмбеддер → re-chunk всего корпуса, не «строка в конфиге».^[2]

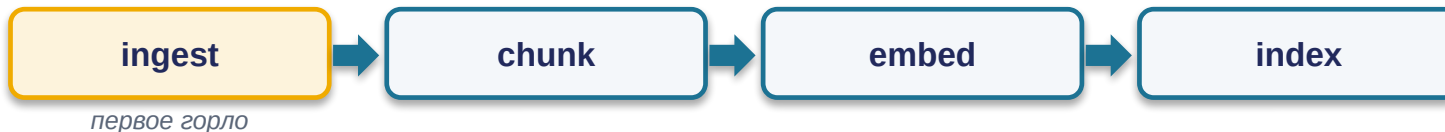


Оценка chunking чувствительна к k

мелкие чанки требуют большего k; меняя нарезку — перепроверяй оптимальный k (Anthropic мерил top-5/10/20).

Прототип → продакшн — это другая архитектура, не подкрутка.

Прототип работает на 1000 документов; ломается на миллионах векторов и тысячах запросов. Проблемы — не только на индексации: и на запросе, и в эксплуатации.



ИНДЕКС-СТОРОНА — сборка и поддержка

Свежесть

батч → устаревание; CDC → субминута ценой ×3 эксплуатации; индекс отстал → тихая деградация^[1].

Смена модели

новый эмбеддер = пере-эмбеддить ВСЕ векторы (старые/новые несравнимы) — миграция.

Оценка

recall@k ~0,8, nDCG^[2] + LLM-судья RAGAS; без набора — деградация невидима.

Стоимость сборки

эмбеддинг + контекст к чанку; препроцессинг ~\$1/1M токенов, разово на масштабе.

QUERY-СТОРОНА + ЭКСПЛУАТАЦИЯ

Качество на масштабе

recall падает и дрейфует с ростом корпуса; «нашёл» ≠ «нашёл правильное».

Латентность / QPS

конкурентные запросы + реранк на запросе; автоскейл опаздывает под пик.

Наблюдаемость

retrieval проваливается тихо, без ошибки — нужны метрики и трассировка запроса.

Безопасность / доступ

кто что вправе извлечь; PII в чанках; фильтр прав ДО поиска.

Каунтерфактуал: база под ~200k токенов целиком кладётся в промпт с кэшированием (до ~90% экономии) — retrieval пропускается вовсе. Первый вопрос перед всей инфраструктурой — «а нужен ли RAG вообще».

[1] Redis — RAG at Scale (ingestion-горло, CDC 24ч → субминута, кэш ~69%) · [2] futureagi — RAG Evaluation Metrics (recall@k ~0,8, nDCG, RAGAS)

«RAG» в вакууме не существует — есть RAG под задачу.

Пять архетипов, у каждого своя природа данных → свой retrieval-дизайн. Общий якорь: наивный dense-RAG промахивается ~40% запросов.

Кейс · система	Ключ дизайна	Провал → чинит
 Поддержка / база <i>kapa.ai · Stripe</i>	гибрид + реранк	dense путает «backoff» и «dead-letter» → BM25 ловит точный код ошибки
 Q&A по документам <i>Vercel docs-copilot</i>	источник ИЛИ отказ	цитирует старую версию API → свежесть + оценка в CI
 Поиск по коду <i>Cursor · Cody</i>	chunk по функции	dense замыливает имя символа → Cody убрал эмбединги → лексика^[3]
 Юридический <i>LexisNexis · Westlaw</i>	гибрид, BM25 для цитат	17–33% галлюцинаций при «0%» ^[2] → верификация цитат + человек
 Корпоративный <i>Glean · eSapiens</i>	роутер к Text-to-SQL	эмбединг таблицы не суммирует → числовой вопрос → SQL

Два кейса — прямые «RAG был не тем инструментом»: Cody вернул классический поиск по коду; агрегирующий вопрос по таблице → SQL, а не «похожие чанки». Retrieval-дизайн выводится из природы данных.^[1]

[1] Barnett et al. 2024 — Seven Failure Points (RAG-архетипы) · [2] Stanford HAI/RegLab — Legal-AI галлюцинирует 17–33% (Lexis/Westlaw) · [3] Sourcegraph Cody — контекст без эмбедингов (классический поиск)

Deep research — это RAG + петля, не «большой промпт».

Определяющий компонент — *retrieval* и оркестрация, поэтому кейс здесь, а не в промптах. Как OpenAI Deep Research, Perplexity, Claude Research.

агент дал отчёт со ссылками	 Panik
часть ссылок даже резолвится	 Kalm
3–13% URL просто сфабрикованы	 Panik



Reasoning-LLM + агентный цикл RAG

план под-вопросов → (поиск → чтение → уточнение рассуждения)×N →
синтез → верификация цитат

Рассуждение направляет поиск, находки уточняют рассуждение — закрытая петля (agentic RAG)^[1], в отличие от статического retrieve-top-k → generate.

Типовой провал — фабрикация цитат

3–13% URL сфабрикованы (в retrieval-augmented режиме); DRACO: верхний результат ~65% качества цитат. Контринтуитивно: deep research хуже обычного поиска — **10,7%** против 4,8% фейковых цитат на запрос, потому что генерирует их кратно больше.^[2]

Против ручного аналитика — быстрее и дешевле, НО без слоя верификации 3–13% ссылок ложны. Любую метрику из ИИ-ресёрча перепроверяй по резолвящемуся первоисточнику; число без базы считай красным флагом, а не фактом.

Когда RAG — правильный выбор.

RAG оправдан при сильном сигнале по признакам ниже^[1] — и отсутствии blockers со следующего слайда «когда НЕ RAG».

Большое / растущее

не влезает в окно целиком, или дорого класть весь корпус в каждый запрос

Меняется

документы, цены, регламенты обновляются чаще, чем выходят версии модели

Свежесть + провенанс

ответ опирается на проверяемый источник; можно показать, откуда факт

Приватная база

знания компании не входят в веса публичной модели

Признаки усиливают друг друга → RAG

Корп. база из тысяч регламентов, обновляется еженедельно, ответ с обязательной ссылкой на пункт: все признаки сошлись, ни один более простой механизм их совместно не закрывает → образцовый профиль RAG.



Один признак — повод присмотреться^[2], но не строить автоматически: сверьтесь с блокерами на следующем слайде.

^[1] IBM — RAG vs Fine-Tuning (вендор-нейтрально) · ^[2] U-NIAH — RAG win-rate выше у меньших моделей

Когда RAG — НЕ правильный выбор.

Знать, когда RAG не нужен, ценнее: это модная архитектура, её ставят туда, где она вредит.

1 Корпус влезает в окно



ориентир — менее ~200k токенов,
меняется редко

→ **полный контекст + кэширование
префикса, не RAG-инфраструктура**

2 Фиксированная политика / значение^[2]



тариф, цена, пункт регламента, правило

→ **детерминированная таблица
соответствий / статическая страница**

3 Данные доступны в реальном времени через API / MCP



во внутреннем сервисе, базе, поиске другой
системы

→ **вызвать инструмент напрямую;
RAG-индекс поверх — лишний, более
хрупкий и более устаревающий слой**



Частый анти-паттерн — ставить RAG-инфраструктуру там, где весь корпус спокойно помещается в контекстное окно и меняется редко.

RAG избыточен, если выполнен ЛЮБОЙ из трёх^[1]: (а) корпус влезает в окно и стабилен, (б) задача сводится к фиксированному значению, (в) знание уже доступно напрямую и живьём через инструмент. «Прямой вызов вернёт данные на момент запроса; RAG-индекс — на момент последней индексации.»

[1] Red Hat — RAG vs Fine-Tuning (когда не RAG) · [2] McCarthy Tétrault — Air Canada («генерация поверх фикс. политики»)

Провал RAG на масштабе.

«Вернул что-то» ≠ «вернул правильное»^[1,2]. У RAG нет сигнала «не нашёл» — он всегда отдаёт к ближайших, даже нерелевантных.



Legal-AI

«ближайшие» дела из другой юрисдикции / отменённый прецедент — модель опирается на них как на факт



Medical-RAG

смешал фрагменты разных пациентов — близки по симптомам, клинически объединять нельзя



Бот поддержки

работал на сотнях статей; после роста до тысяч качество тихо просело — никто не заметил

Air Canada — разбор архитектуры^[3]

Диагноз

сгенерированный правдоподобный текст поставлен в роль, требовавшую извлечённого проверенного факта — отказ опоры на источник

Правильная альтернатива

фиксированная политика → таблица соответствий / страница; нужен диалог → RAG со строгой опорой на источник, обязательная цитата, явное «не знаю», проверка человеком

РАЗДЕЛ 3

Дообучение vs промпт vs RAG

Проблему знания мы решили через RAG. А если проблема не в знании, а в поведении модели — её тоном, форматом, политикой?

настройка поведения · 4 разбора · 1 провал

Раздел 0
Открытие

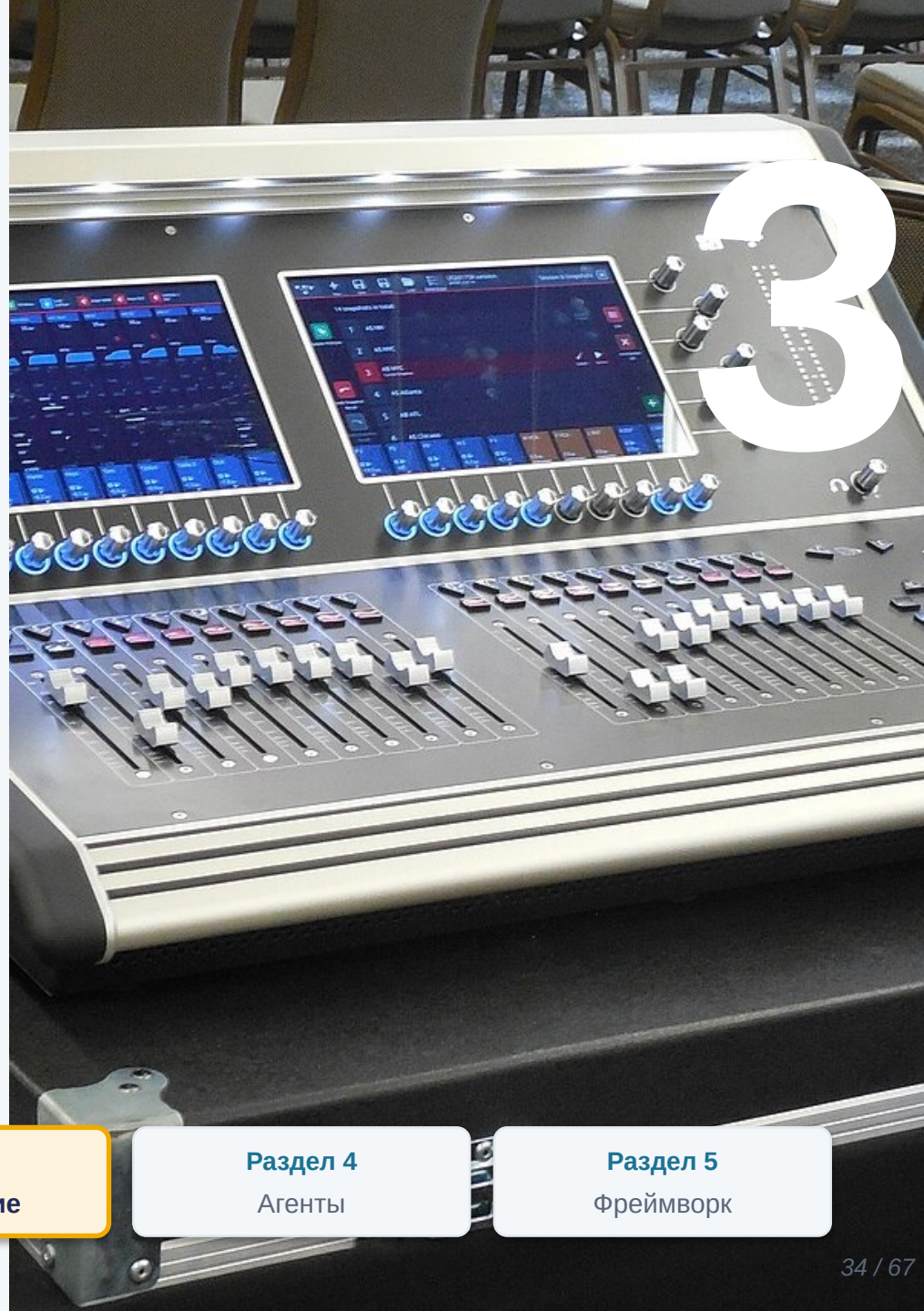
Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Дообучение

Раздел 4
Агенты

Раздел 5
Фреймворк



Что такое обучение модели: train/val/test и transfer learning.

Обучение под задачу — это размеченная выборка с эталоном, разбиение train/val/test и перенос обучения (transfer learning) поверх предобученной модели. На этом стоит дообучение.



Выборка + эталон

Размеченный набор «вход → правильный ответ» (ground truth, эталонная разметка); модель учится его воспроизводить и обобщать на новые данные.



Train / val / test split

Три непересекающиеся части: на train учат, на validation подбирают, на test меряют один раз. Правило: нельзя тестировать на обучающих данных.



Transfer learning (перенос обучения)

Взять предобученную на большом корпусе модель и дешево дообучить под свою узкую задачу: быстрее и точнее, чем с нуля («предобучение → дообучение»).

ЧТО ОСТАВИТЬ

Наборы для оценки (golden set) — без них катастрофическое забывание не видно; версионирование данных и весов для отката; train/test-дисциплину против утечки; мониторинг дрейфа. LoRA удешевила шаг дообучения, но не дисциплину вокруг него.

Мост: PEFT/LoRA — тот же transfer learning, доведённый до предела дешевизны и поверх несравнимо более крупной модели. Идея не новая — новыми стали масштаб базовой модели и стоимость шага.

Что такое дообучение (fine-tuning).

Дообучение (fine-tuning) — продолжение обучения уже готовой модели на ваших данных. В Л1 — тип использования; здесь — архитектурный выбор, одна из ступеней лестницы.



Промпт / RAG → КОНТЕКСТ

Меняют только вход — веса не трогаются; эффект живёт лишь в рамках запроса.

Дообучение → САМИ BECA

Изменение встроено в модель — действует всегда и стоит дороже того, что меняет контекст.

Промпт/RAG = «что показать модели». Дообучение = «изменить саму модель»^[1]. На практике «дообучить» почти всегда означает LoRA/PEFT, а не переобучение всех весов (следующий слайд).

[1] IBM — RAG vs Fine-Tuning (fine-tuning меняет веса)

Стоимость обучения: важен порядок и характер роста, не точная цифра.

Значения относительные: Full-FT взят за базлайн ×1 по VRAM / \$ / компьютеру, остальные — доли от него. Абсолютные \$ и часы движутся; смысл — порядок и характер роста.

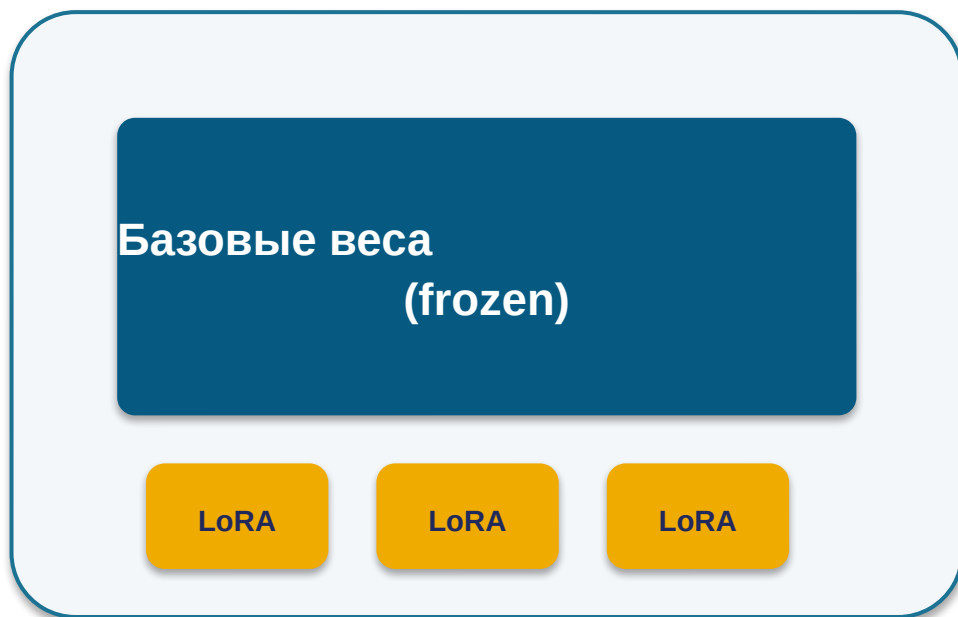
Ось	Претренин с нуля	Full-FT	LoRA	QLoRA	Промпт+RAG
Обучаемые параметры	100% (с нуля)	100% весов	~0,1–1%	~0,1–1%	0% — веса не меняются
VRAM (память)	кластер	×1 (базлайн)	~×0,2	~×0,05	только инференс
\$ за прогон	×1000+	×1 (базлайн)	~×0,01	~×0,01	~0
Компьютер (GPU-часы)	×1000+	×1 (базлайн)	~×0,01	~×0,01	~0
Объём данных	триллионы токенов	тысячи–десятки тыс.	тысячи–десятки тыс.	как LoRA	few-shot (3–20)
Скорость итерации	месяцы	медленно	быстро	быстро	мгновенно

LoRA — дефолт: обучает ~0,1–1% параметров, ~×0,01 по цене / компьютеру и ~×0,2 по VRAM от Full-FT — 7B-LoRA <\$10 — дешевле фронтпретрейна (\$61–92M) на ~7 порядков^[1]. Full-FT оправдан лишь когда датасет превышает ёмкость LoRA — иначе только ×5–6 памяти и риск забывания. QLoRA — тот же LoRA поверх 4-битной базы: 65B влезает в одну 48-ГБ карту^[2].

[1] Thinking Machines — LoRA Without Regret (LoRA ≈ Full-FT при условиях) · [2] Dettmers et al. 2023 — QLoRA (65B на одной 48-ГБ карте) · Относительные величины, Full-FT = базлайн ×1; конкретные \$ и часы движутся — важен порядок и характер роста, не точная цифра.

PEFT вместо полного дообучения.

PEFT — базовые веса замораживаются, обучается лишь небольшой набор адаптеров. LoRA — низкоранговые матрицы-адаптеры; QLoRA — то же поверх квантованной модели^[1,2].



адаптеры — мегабайты vs гигабайты

1. Дешевле и быстрее

миллионы параметров вместо миллиардов; QLoRA — на одном GPU

2. Модульность

адаптеры мегабайты vs гигабайты; одна база — много специализаций

3. ↓ Риск забывания

база заморожена, физически не переписывается под новый сигнал



PEFT / LoRA: дешево, модульно, ниже риск забывания



Полное дообучение всех весов в 2026 — почти никогда

98,4% моделей с тегом PEFT — это LoRA^[3]

из 20 834 карточек на Hugging Face Hub · оговорка: доля среди моделей с тегом PEFT, не среди всего дообучения

PEFT (LoRA/QLoRA) почти всегда предпочтительнее полного дообучения: дешевле, модульнее, ↓ риск забывания (forgetting). Полное дообучение в 2026 — почти никогда.

Что здесь знание, что поведение, что детерминировано.

Вопрос на проектировании — не «RAG или дообучение». Разнесите проблему по осям: знание → RAG, поведение → PEFT, дешевле → дистилляция, детерминированное → код.

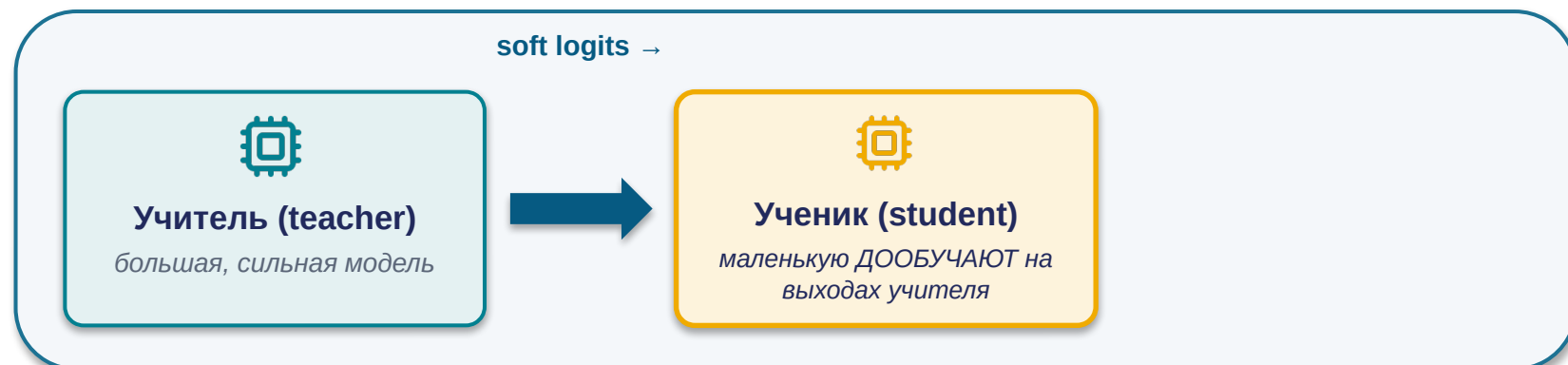
Если задача требует...	→ правильный инструмент	...и НЕ этот, потому что
знание меняется / нужны свежесть, провенанс	RAG (или длинный контекст для малого корпуса)	не дообучение: знание устареет, переобучать дорого, риск забывания
стабильное поведение / тон / формат / политика	дообучение (PEFT)	не RAG: подаёт знание в контекст, но не меняет манеру модели
снизить стоимость / задержку на узкой задаче	дообучить учителя + дистилляция в ученика	две отдельные техники в связке; промпт/RAG не уменьшают размер модели
детерминированный, верифицируемый ответ	обычный код, без ИИ	ни RAG, ни FT: ИИ добавит недетерминизм без выигрыша

Что делать: гибрид — норма^[1] TAM, где у задачи есть И проблема знания, И проблема поведения. Нет проблемы поведения — FT не нужен, даже когда есть RAG. Каждый компонент добавляется под своё требование — то же правило лестницы.

[1] [BigData Boutique — Fine-Tuning When RAG Isn't Enough](#)

Дистилляция — это дообучение маленькой модели: отличается целью и данными, не тем, что «не обучение».

Ученика действительно дообучают, поэтому «это дообучение» и «это дистилляция» истинны одновременно. Отличие от обычного fine-tuning — не в «обучают/не обучают», а в двух осях ниже.



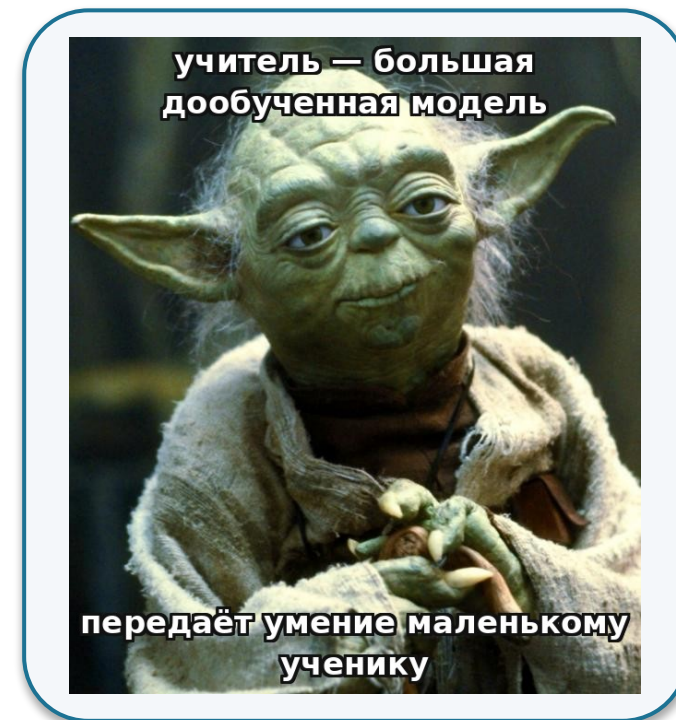
Ось 1 — ЦЕЛЬ (что матчим):

обычный SFT учит на жёстких метках «вот правильный класс». Дистилляция матчит РАСПРЕДЕЛЕНИЕ учителя^[1] — мягкие вероятности (dark knowledge: «кошка похожа на собаку сильнее, чем на грузовик»). Минимизируют KL-дивергенцию.

Ось 2 — ДАННЫЕ (откуда сигнал):

обучающий сигнал генерирует САМ учитель, не человек-разметчик. Response-distillation (тексты-ответы, Alpaca-стиль) работает даже без доступа к логитам — через API. DistilBERT: −40% размера / 97% качества^[2]. Alpaca: 52k инструкций от GPT, <\$600.

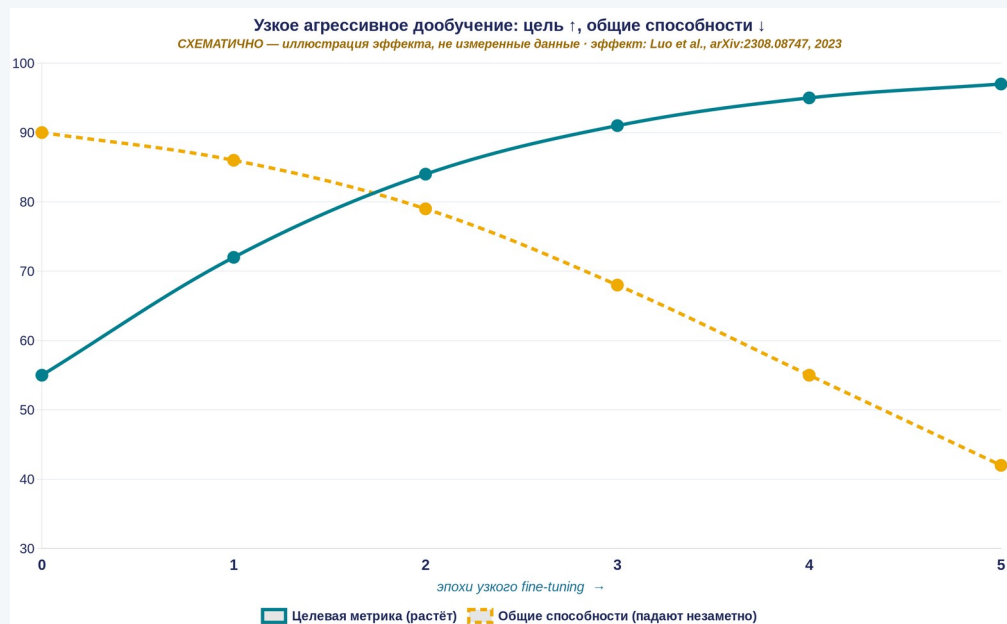
Провал: у дистилляции жёсткий ПОТОЛОК способностей — ученик почти никогда не превосходит учителя (тот задаёт верхнюю границу качества) и НАСЛЕДУЕТ его ошибки и смещения, но без способности учителя «поймать себя». Дистиллируете GPT-3.5 — получаете не-лучше-GPT-3.5.



Правильно: «дообучить компактную student-модель на выходах сильной teacher» — это и дообучение, и дистилляция сразу.

Провал: катастрофическое забывание.

Катастрофическое забывание (catastrophic forgetting) — деградация общих способностей модели в результате узкого агрессивного дообучения^[1].



тяжелее с ростом масштаба модели — у крупной выше исходный уровень, ей «больше падать»

Нет петли проверки (eval) на общих задачах + нет версий датасета/весов → не увидишь поломку до боевой эксплуатации + не сможешь откатиться → это не «риск», это критерий «НЕ делай дообучение»

Правильно: PEFT (замороженные веса — ниже риск); для меняющегося знания — RAG, не FT вовсе.

[1] Luo et al. 2023 — Catastrophic Forgetting in LLM Continual FT · Эмпирически наблюдается при постоянном дообучении (Luo et al. 2023). Механизмы — «исследования показывают» (препринт).

Оценка обучения дороже самого обучения — и ни один из шести методов не самодостаточен.

Полезный 7B-LoRA стоит <\$10 — а строгая оценка того же адаптера — \$300–1200 человеко-труда за раунд и всё равно может не предсказать продакшн. Каждый метод — по схеме «как работает · когда валиден · когда ломается · стоимость».

Метод	Как работает	Когда ломается (число)	Стоимость
Публичный бенчмарк	точность по фикс. набору Q&A (MMLU/GSM8K/HELM)	контаминация: MMLU ~29% вопросов ^[1] ; GSM8K → GSM1k –13 п.п. ^[2] — память, не рассуждение	низкая (обманчиво)
Отложенный (held-out) тест-набор	заморозил ДО тренировки, гоняешь каждую версию	утечка в обучающую выборку / малый объём / дрейф от продакшна	перв. труд, ≈0 повтор
LLM-как-судья	сильная модель судит; парно > поштучно	>80% согласия с людьми (= человек-человек), НО позиция до 75%, самозавышение (self-enhancement) +10–25%	центры + контроль смещения
Оценка людьми	критериальная шкала + ≥2–3 аннотатора + к-согласие	к≥0,6 нужен (иначе шкалу переписать); дорого, не масштаб	\$300–1200 за раунд
Метрики задачи (BLEU/ROUGE)	перекрытие n-грамм с эталоном	на открытой генерации r≈0,25–0,52 с людьми — награждает n-грамму, не смысл	почти ноль
A/B / онлайн	canary 1–5% + guardrail-метрики на живом трафике	нет трафика до прода; ×4 сэмплов из-за недетерминизма	инженерия + риск

Ни один метод не самодостаточен: дешёвые (бенчмарк, BLEU) — низкая валидность; валидные (отложенный-эксперт, оценка людьми, A/B) — дорогие. Отложенный тест-набор даёт наибольшую отдачу, строишь ДО тренировки. Разрыв лаборатория → продакшн ~37% — механика, не аномалия: прирост на бенчмарке — гипотеза, а не результат; подтверждают только чистый отложенный набор + A/B на реальном трафике.

[1] [Pebblous — LLM Benchmark Contamination \(MMLU ~29%\)](#) · [2] [Zhang et al. 2024 — GSM1k \(чистое зеркало GSM8K –13 п.п.\)](#)

РАЗДЕЛ 4

Агенты

От собеседника в окне чата — к компоненту боевой системы: цикл, экипировка, память, доступ к инструментам — и где всё это ломается.

агент + экипировка · 5 разборов · 4 провала

Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Дообучение

Раздел 4
Агенты

Раздел 5
Фреймворк



Что такое управляющий цикл: конечный автомат, workflow-движки, OODA.

Управляемая автоматизация — это конечный автомат состояний, workflow-движки по фиксированной схеме и управляющий цикл *plan → act → check* (OODA). На этом стоит агент.



Конечный автомат

Набор состояний и правил перехода по событиям («создана → в обработке → закрыта»). Возможные переходы видны, невозможные исключены конструктивно.



Workflow-движки

Исполняют заранее описанный процесс по фиксированной схеме: BPMN, DAG-оркестраторы (Airflow), RPA. Порядок шагов определён заранее, а не на лету.



Управляющий цикл

plan → act → check как контур обратной связи (теория управления, АСУ); военный аналог — OODA-петля. Не одно действие, а цикл с коррекцией.

ЧТО ОСТАВИТЬ

Детерминированные workflow где можно; идемпотентность (повтор шага не ломает состояние); принцип наименьших привилегий (*least-privilege*) для инструментов; лимиты и аудит цикла. Практики надёжности контура никуда не деваются.

Мост: LLM-агент — тот же управляющий цикл *plan → act → check*, но шаги в нём генерирует модель недетерминированно, а не выбирает фиксированное правило автомата. Отсюда и гибкость, и провалы раздела.

Цикл агента: план → действие → проверка → повтор.

Агент — архитектура, где модель не делает один проход, а работает в цикле^[1], сама определяя последовательность шагов.



[1] Yao et al. 2022 — ReAct (plan → act → check → iterate) · [2] Anthropic — Reasoning Models faithfulness (check ≠ самооценка) · Паттерны цикла (ReAct, Reflexion, Plan-and-Execute) — в главе. Проектировать агента = проектировать защиту на каждом из 4 шагов.

Модель становится компонентом системы.

В L2 модель была собеседником в окне чата. Здесь она — вызываемый компонент: получает запрос через API, формулирует «вызови инструмент X», а исполняет ваш код система, не модель. Это сдвиг от «поговорить» к «встроить».

>_ Модель как компонент + tool use

Модель решает «нужен инструмент» и формулирует вызов; исполняет — ваш код. Модель не выходит наружу сама — она возвращает намерение, а система выполняет и возвращает результат в контекст.

Это и есть шаг «действие» цикла агента: модель в петле «рассуждать → вызвать инструмент → прочитать результат».

MCP — «USB-C для инструментов LLM»

N×M несовместимых интеграций → **N+M**^[1]

*инструмент один раз = MCP-сервер; модель один раз = MCP-клиент.
Подключение стало операцией на минуты.*

AI Anthropic 11/2024 →  OpenAI 03/2025 →  Google 04/2025

Что это меняет: модель встраивается через structured output и tool use, а MCP превращает N×M интеграций в N+M.

- structured output + tool use: модель возвращает намерение, код исполняет

- MCP — стандарт подключения; его безопасность — на слайде про MCP

Модель-компонент и MCP делают подключение удобным, но ни один механизм не делает модель надёжнее — правило лестницы не отменяется. Удобство подключения — не аргумент за подключение.

[1] Anthropic — MCP donation / Agentic AI Foundation (N×M → N+M) · Масштаб экосистемы MCP и хронология поддержки вендорами — см. источники.

Агент — это не «чат подороже». Это другой порядок цены.

База сравнения — один chat-turn (десятки центов). Каждая ступень вверх умножает расход токенов, а не прибавляет его.

Один вызов LLM (chat)

базовая стоимость запроса

×1

Одиночный агент (цикл)

план → действие → проверка → повтор: много проходов на задачу

≈50×

Мульти-агент

поверх агента — координация нескольких агентов (Anthropic, 2025)

ещё ×15

Что делать: считайте бюджет ДО выбора архитектуры. Кэш промптов (не пересчитывать неизменный префикс) в агенте — уже не «приятный ориентир», а необходимость: без него ×N-стоимость цикла становится неподъёмной.



Множители — порядок величины (Anthropic, 2025), не точный тариф; абсолютная цена зависит от модели и задачи.

МСП: подключить инструмент — теперь операция на минуты.



МСП — стандарт подключения (уже разобран выше)

Один раз описал инструмент как МСП-сервер — и его видят все модели-клиенты. Подключение стало операцией на минуты. Но лёгкость подключения ничего не говорит о том, что именно подключаешь.

Читайте каталог критически

«до 90 000 серверов» → реально запускаемых $\approx 10\,000 = 11\%$ каталога.

Остальное — дубли, битые и заглушки. И даже рабочие 10 тысяч не проверены на безопасность.



Поворот доверия: стандартизация подключения $\neq$ безопасность подключаемого — и обостряет её.

30+ CVE

за 60-дневное окно против МСП-серверов (~43% — внедрение команд)

82%

обход пути (path traversal) среди 2 614 проверенных реализаций

Что делать: удобство подключения — не аргумент за подключение. Каждый МСП-сервер = чужой код в вашем окружении + носитель инъекции в контексте. Подключаете под требование задачи, с оценкой новой границы доверия.

МСП или прямой API — это не альтернативы: МСП окупается как флот, ниже порога это лишний слой.



База: REST/gRPC — транспорт, который делает работу (ходит в базу, дёргает сервис). **МСП — тонкий стандартный слой ПОВЕРХ**, позволяющий агенту во время работы обнаружить (`tools/list``) и вызвать инструменты. В большинстве развёртываний МСП-сервер **ОБОРАЧИВАЕТ** уже существующий REST API, а не заменяет его. Порог: выигрыш $N+M < N \times M$ материализуется только когда $N, M \geq 2-3$.

Взять МСП, когда...

- ✓ много агентов × много инструментов ($N, M \geq 2-3$)
- ✓ нужно обнаружение инструментов во время работы (каталог меняется)
- ✓ нужна переносимость между вендорами моделей / хостами
- ✓ серверы переиспользуют другие команды / третьи стороны
- ✓ готовы провести security-ревью каждого сервера

Взять прямой API / function-calling, когда...

- одно приложение, один-два известных инструмента ($N+M \geq N \times M$)
- набор инструментов фиксирован на этапе сборки
- вы зафиксированы на одной модели, одном стеке
- интеграция приватная, единственный потребитель
- нужна маленькая аудируемая запертая поверхность СЕЙЧАС

~25%

серверов реестра непригодны» (пол, не потолок)

43,7%

верхний результат на MCP-Universe» (>56% задач провалено)

+риск

injection / path-traversal — см. слайд про МСП (поворот доверия)

Правило: МСП — стандарт интероперабельности, а не апгрейд производительности. Пропустить МСП для одного агента с одним инструментом — правильное инженерное суждение, не срезанный угол (то же правило лестницы, §5.1).

Workflow — не примитив: пять паттернов вокруг предсказуемого.

Прежде чем тянуться к динамическому агенту: большинство «нам нужен агент» закрываются одним из этих пяти паттернов. Все они — предопределённые в коде структуры, сохраняющие предсказуемость и аудируемость.



Все пять — предопределённые в коде структуры; динамический агент отличается тем, что сам решает, какую структуру построить под конкретный вход, и теряет предсказуемость, которую эти паттерны сохраняют. Найди простейшее: workflow — вокруг предсказуемого, агент — чтобы исследовать непредсказуемое.

Фреймворк агента — абстракционный налог; у всех пять общих минусов.

Anthropic: многие паттерны — несколько строк прямых вызовов API^[1]; фреймворки скрывают исходные промпты и усложняют отладку. Сначала пять минусов, общих для ВСЕХ, потом разбор по схеме.

 **Кастомизация**
долг за пределами «счастливого пути»

 **Прозрачность**
прячет реальные промпты и поток управления

 **Привязка (lock-in)**
SDK вендоров привязывают к экосистеме

 **Смена версий**
ломающие API-изменения (LangChain)

 **Накладные расходы**
CrewAI +18% токенов vs LangGraph[®]

Фреймворк	Ядро-абстракция	Под что подходит	Один честный минус
LangGraph	stateful-граф	продакшн: поток управления, состояние, human-in-the-loop	крутая кривая; для одного цикла — переинжиниринг
CrewAI	ролевые «экипажи»	быстрый роль-декомпозируемый прототип	роль маскирует, что реально происходит; хрупкость мульти-агента
AutoGen → AG2	разговор агентов	исследование разговорных паттернов	частые переименования (AutoGen → AG2 / MS Agent) — риск при внедрении
OpenAI Agents SDK	handoffs (передача)	лёгкая передача задач + трассировка на OpenAI	OpenAI-центричность; сохранение состояния/RAG приносишь сам
Claude Agent SDK	тот же loop, что Claude Code	coding / computer-use, длинные задачи	Claude-центричность; тяжёлый harness vs голый API
smolagents	code-агенты (пишут Python)	минимальный читаемый агент	исполнение кода = поверхность атаки (нужна песочница)
LlamaIndex agents	агент поверх RAG-стека	агент над вашими документами/данными	центр тяжести — retrieval, не оркестрация
Pydantic AI	type-safe agent loop	валидированные выходы, типовые контракты	молод; type-safety ортогонален оркестрации
НИ ОДНОГО	обычный код + прямые вызовы API	детерминированные шаги, ≤ пары вызовов LLM	требует дисциплины «не тянуться к фреймворку авансом»

Пять минусов выше применимы к КАЖДОЙ строке. Нижняя строка — не для полноты: детерминированный путь + известные шаги + чувствительность к стоимости → обычный код, без агента и без фреймворка. Правило лестницы на уровне выбора инструмента: сложность оснастки оплачивается требованием задачи, а не берётся авансом.

[1] Anthropic — Building Effective Agents (фреймворк = абстракционный налог) · [2] LangGraph (LangChain) — stateful-граф, продакшн-оркестрация

Агент — под открытую задачу; мульти-агент нужен только в трёх условиях.

Сверху вниз: предсказуемо → workflow; непредсказуемо и ценно → один агент; несколько агентов — только под конкретный кейс. Каждый лишний шаг перемножает вероятность успеха, а не усредняет.



1. Предсказуемо → workflow

шаги известны заранее; предопределённые в коде пути, аудируемость



2. Непредсказуемо → один агент

шаги зависят от промежуточных результатов; цена оправдана ценностью



3. Мульти-агент — под конкретный кейс

нужен только в 3 условиях (справа); вне их — неверный инструмент

Три условия, когда мульти-агент оправдан:

1. Широко-параллельные независимые ЧТЕНИЯ (не решения)

напр.: прочитать 100 источников параллельно и свести в отчёт (Anthropic research).

2. Независимые перспективы для кросс-проверки

напр.: 5 агентов независимо ищут → голосование.

3. Изолированные под-домены с отдельными инструментами / правами

напр.: отдельный агент на биллинг, отдельный на инфраструктуру.

Cognition vs Anthropic — примирение:

Anthropic: мульти-агент выигрывает research-задачи. Cognition: «не стройте мульти-агентов». Противоречия нет: параллельте независимые ЧТЕНИЯ (поиск вширь), но не РЕШЕНИЯ с зависимостями. Топология: рой связей = быстрее коллапс».

Вне этих кейсов мульти-агент — неверный инструмент: надёжность падает как r^n ($\times 15$ токенов). Числа и замеры — на следующем слайде.

Начинай с одного сильного агента. Мульти-агент оправдан только в трёх кейсах выше и когда ценность оправдывает кратную цену; иначе это неверный инструмент.

Мульти-агент по умолчанию — не выигрыш, а множитель риска.

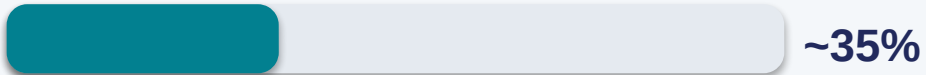
В цепочке из n шагов, где сбой любого портит результат, надёжности перемножаются: успех $\approx p^n$. Инженерная интуиция «каждый шаг почти всегда работает» математически ложна.

Надёжность цепочки = p^n

95% × 10 шагов



90% × 10 шагов



95% на шаг звучит надёжно — но $0,95^{10} \approx 0,60$, $0,95^{20} \approx 0,36$. Больше агентов = больше шагов = ниже общая надёжность.

t-bench: p^n , измеренный вживую

GPT-4o **61%** pass@1 → **25%** pass@8: стабильно на всех восьми прогонах лишь в четверти случаев.^[1]

Топология решает: координатор > рой

«Рой» равноправных агентов амплифицирует ошибки **17,2×**^[2,3], один координатор — только **4,4×** (Kim et al., 2512.08296).



Anthropic: выигрыш «because it helps spend enough tokens» — от объёма токенов, не от «магии координации».

Что делать: начинай с одного сильного агента. Цена растёт кратно — одиночный $\approx 4\times$ токенов чата, мульти $\approx 15\times$. Мульти-агент — только если задача распадается на **ШИРОКО** параллельные независимые подзадачи высокой ценности; иначе $+15\times$ токенов и координационные издержки не окупятся.

Каждый слот экипировки агента — компромисс, не выигрыш.

Реальный агент-помощник (Claude Code, Cursor, Aider) — это цикл план → действие → проверка → повтор ПЛЮС оснастка. Пять типовых слотов^[1]:



Как не поднимаешься по лестнице архитектур без требования задачи — не добавляй агенту память, субагентов или MCP «на всякий случай». Каждый слот несёт свою цену: операционную сложность, новую поверхность отказа, новую границу доверия — и должен отвечать на конкретный триггер (Раздел 5).

[1] agent-harness-registry — карта слотов экипировки агента

Специально организованная память агента — не всегда во благо.

В пределах одной сессии агент «помнит» всегда — это контекст. Речь о ДОБАВЛЕННОМ постоянном слое памяти между сессиями (mem0 / Cognee / Letta / Memory Tool): наличие такого слоя интуитивно кажется чистым выигрышем. Независимая проверка показывает: иногда — драматически нет^[1].



Letta — уровень D (Tier D)

проигрывает И голой модели, И плоскому файлу на всех задачах.
persistbench_v1:

Голая модель	1.000	94 с
Плоский файл	0.833	159 с
Letta	0.750	496 с

Механизмы: капитуляция под давлением · многословие топик факт · факт замечен, но не закодирован.

Оговорка: тест на Letta v0.6.7 — отстаёт от текущей v0.16.8 (~18 мес).

Anthropic Memory Tool — уровень B (Tier B)

В целом сильный результат — но даже он теряет данные в

17% задач

- явный отказ записать «эфемерный» факт
- немотивированный отказ «вне рамок»
- тихое сжатие с потерями — деталь не восстановить
- невоспроизводимость: та же беседа дважды → разный результат

[1] agent-harness-registry (live-eval) — Letta Tier D / Memory Tool 17%-хвост · agent-harness-registry (workain lab, независимая проверка 2026-07-05). «Работает хорошо» ≠ «работает всегда» — тот же урок, что RAG на масштабе и забывание.

Навыки, субагенты, доступ — и граница доверия.



Навык (skill)

переиспользуемая процедура «как делать»
под повторяющуюся задачу



Субагент

отдельное контекстное окно: не засорять
основной + изолировать недоверенное



MCP-доступ

каждое подключение — новая граница
доверия и политика хранения



Безопасность: как только агент делегирует и подключается — появляется поверхность ^[2,3], которой не было у одного вызова

GitHub MCP: утечка через инъекцию (май 2025)

issue с встроенной инструкцией + переизбыточный токен (PAT на все репо)
→ ассистент выгрузил приватные репозитории в публичный PR^[1].

Катастрофа = инъекция × широкие права. Убери любое — атака не проходит.

«У нас ZDR» ≠ «нигде не хранится»: судебный приказ (NYT v. OpenAI) + сторонние сервисы/MCP вне ZDR^[4,5]. Регулируемое — не слать без ZDR/BAA.

4 правила проектирования

- 1 Наименьшие привилегии (least-privilege)**
минимум токенов/прав
- 2 Изоляция недоверенного**
отдельно от привилегий (субагент)
- 3 Человек в контуре на запись**
необратимое — только через человека
- 4 Белый список / фиксация версий**
аудированные версии; запрет по умолчанию

«У нас ZDR» ≠ «данные защищены во всей цепочке».

ZDR (zero data retention) покрывает основные вызовы модели — но не всю архитектуру. Агент = модель + инструменты, и инструменты часто вне ZDR.



ZDR НЕ покрывает

- Files API
- пакетную обработку
- исполнение кода в контейнерах
- MCP-коннектор
- сторонние интеграции
- потребительские планы

Чем агентнее архитектура — тем больше её данных проходит через эти звенья.



Судебный приказ переживает вашу политику

NYT v. OpenAI (2025): суд обязал сохранять ВСЕ логи ChatGPT как доказательства. Договорная политика «30 дней» оказалась бессильна против litigation hold по чужому спору.

Данные, покинувшие ваш периметр, живут по правилам, на которые вы не влияете.

Что делать: составьте карту данных по каждой функции до боевой эксплуатации — какие данные, через какое звено, с какой политикой хранения, какие звенья сторонние. Регулируемые/чувствительные данные — только с ZDR/BAA или локально (on-prem).



Реальные агенты для кода — через рамку экипировки.

Разница между инструментами — не в «качестве модели», а в том, какие слоты экипировки заполнены и где агент физически живёт^[1].

<> Claude Code

терминал + IDE · проприетарный

широкая экипировка: память, файлы-инструкции, навыки, полноценные субагенты, MCP — почти все 5 слотов

цена — большая операционная сложность

<> Aider

сначала терминал · открытый код

минимальная простота: без развитой памяти, без субагентов, без навыков. ~47k★ на GitHub

«тонкая» оснастка — самостоятельный выбор, не недоразвитость

<> Cursor

настольная IDE (форк VS Code) · проприетарный

агент внутри редактора — не терминал. Форма интеграции — отдельная ось от оснастки

где живёт агент — тоже архитектурное решение

<> OpenHands

платформа на своём сервере · MIT · ~80k★

широкая автономность, локальное/Docker/облако развёртывание

рабочая гипотеза по совпадению профиля — вероятный кандидат на «OpenClaw», не подтверждённый факт

Выбор инструмента подчиняется тому же правилу: не бери переоснащённый по умолчанию — смотри, какая оснастка нужна именно твоей задаче.

[1] [agent-harness-registry](#) — обзор через рамку экипировки

Шесть типовых агентных задач по одной схеме: где агент оправдан, а где часть задачи стоит откатить в workflow.

Задача	Форма петли	Где ломается (число / кейс)	Лучше workflow?
Coding-агент (Claude Code / Cursor)	читать репо → план → правки → тесты → PR	большие рефакторинги; тихие неверные правки; петля на «мигающих» тестах	Нет — открытый поиск по коду требует агентности
Клиентская поддержка (Air Canada · Klarna)	классифицировать → достать политику → решить/эскалировать	Air Canada: бот выдумал политику → трибунал взыскал. Klarna: «~700 агентов» → вернули людей	Часто ДА для риск-части: детерминированная маршрутизация + guardrails
ETL / data-pipeline	заметить дрейф схемы → пропатчить → провалидировать → применить	тихие плохие преобразования на масштабе; недетерминизм в аудируемом пайплайне	В основном ДА: ствол детерминирован, агент — на край починки дрейфа
Research-агент (Deep Research)	план под-вопросов → веером искать → синтез → цитаты	взрыв стоимости ~15× токенов; галлюцинированные цитаты; поверхностная агентность	Нет для открытого «вширь»; ДА если это фиксированный поиск по ключу
Браузер / Operator	скриншот/DOM → план UI-действия → клик → наблюдать	OSWorld ~38% (1-е поколение); ~1 задача из 5 проваливается; Operator ЗАКРЫТ	Для значимых действий (платежи) ДА — используй реальный API
SRE / ops-агент (ITBench)	телеметрия → гипотеза → дашборд → предложить фикс	ITBench: ~14% SRE-сценариев решаются автономно — сузить поиск, не заменить человека	Для ИСПРАВЛЕНИЯ ДА: workflow + подтверждение; агент — на расследование

Сквозной принцип: агент — на адаптивный КРАЙ (открытый поиск, починка дрейфа, расследование), workflow-с-воротами — на СТВОЛ и на значимое действие. Числа Operator (~38%) и ITBench (~14%) означают «сузить пространство поиска», а не «заменить человека».

[1] McCarthy Tétrault — Air Canada (бот выдумал политику) · [2] OSWorld / WebArena / ITBench — бенчмарки агентных задач · \$4.9b; OSWorld / WebArena / ITBench — verify day-of. Отдельные суммы/детали — иллюстративны; классы и привязки (Air Canada, Klarna, Operator) реальные.

Провалы агентов — это КЛАССЫ, а не список курьёзов.

Из **344** значимых для бизнеса ИИ-инцидентов в **188** (~55%) автономная система нанесла ущерб в бою БЕЗ атакующего — агент сломал всё сам. Запоминайте класс, не дату.

Класс провала	Кейс · дата	Выученный урок
Деструкция + избыток прав	PocketOS · 04.2026	Системный промпт ≠ контроль; жёсткая граница + наименьшие привилегии
Инъекция без клика (zero-click)	EchoLeak / M365 Copilot	«Смертельная тройка» = эксплуатируемо
Неуправляемый расход	\$48k/14ч · \$1,3M/30дн	Нет критерия успеха и жёсткого потолка → цикл не завершится
Галлюцинация пакетов	slopsquatting · 19,7%	Выдуманное имя пакета → атакующий его регистрирует; не доверять без реестра
Вредоносный MCP-сервер	postmark-mcp · 09.2025	MCP-сервер = непроверенная цепочка поставок; подмена версии (rug-pull)
Мульти-агентный каскад	61% каскадов из апстрима	Где сломалось ≠ где проявилось; ошибки перемножаются
Юр. ответственность	OLG Hamm · Air Canada	«Это ответил бот» — не защита; вывод принадлежит компании
Петля без бюджета	\$4 200 / 63 ч	«Пробуй, пока не выйдет» без лимита = буквально
Накопление ошибок	надёжность как р ⁿ	р ⁿ , а не усреднение; «докрутить модель» — слабый рычаг
Мульти-агентная хрупкость	Cognition, 2025	Для задач с зависимостями мульти-агент хуже одного

Что делать: столкнувшись с новым инцидентом — назовите ЕГО КЛАСС. Класс задаёт контрмеру (лимит / наименьшие привилегии / проверка между шагами), а конкретные даты и суммы — не нужно.

Четыре класса крупным планом — с базой сравнения.



RocketOS — боевая БД за 9 секунд

Агент нашёл токен с неограниченными правами в чужом файле и удалил том + все резервные копии. Ближайший восстановимый — 3-месячной давности.

Урок: системный промпт — не средство контроля безопасности; нужна жёсткая граница.



Неуправляемый расход: \$48k / 14ч · \$1,3М / 30дн

Запрос без критерия успеха — планировщик расширился и не завершился. База: обычная сессия — центы-доллары; это на 3–5 порядков выше.

Урок: критерий завершения + жёсткий потолок + аварийный стоп обязательны.



Slopsquatting — 1 из 5 импортов

На 576 000 примеров кода 19,7% ссылок на пакеты — галлюцинации. Атакующий регистрирует выдуманное имя → следующий агент ставит чужой код.

Урок: фиксация (pin) версий/хешей, lock-файлы, проверка новых зависимостей.



Каскад — 61% ошибок из апстрима

На 73 инцидентах: в 61% корень был в апстрим-слое (поиск/план), а не там, где сбой стал заметен. Следующий агент принимает ошибку за факт.

Урок: проверка между шагами, меньше переходов, трассируемость.

Что делать: во всех четырёх корень один — агент применён без внешней границы (прав, бюджета, валидации). Граница ставится ВНЕ агента, промптом её не заменить.

РАЗДЕЛ 5

Как выбрать: фреймворк решения

Мы разобрали все архитектуры по отдельности — и где каждая проваливается. Теперь соберём это в один инструмент выбора.

инструмент выбора · 4 разбора

Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Дообучение

Раздел 4
Агенты

Раздел 5
Фреймворк

5



Что такое дисциплина выбора: KISS, YAGNI, наименьшая мощность.

Дисциплина выбора — это отбор технологии от требования, а не от инструмента: KISS, YAGNI и принцип наименьшей мощности. На них стоят лестница и чек-лист раздела.



От требования, не от инструмента

Требования-инжиниринг: сначала фиксируют, что система обязана делать, потом подбирают инструмент. Build-vs-buy: строить своё или взять готовое.



KISS + YAGNI

KISS (Keep It Simple, Stupid — «делай проще»): простейшее решение под требование. YAGNI (You Aren't Gonna Need It — «не понадобится»): не закладывай мощность «на будущее».



Наименьшая мощность

Из заметок W3C (Бернерс-Ли и Мендельсон): бери наименее мощный из достаточных инструментов — его проще анализировать, проверять и сопровождать.

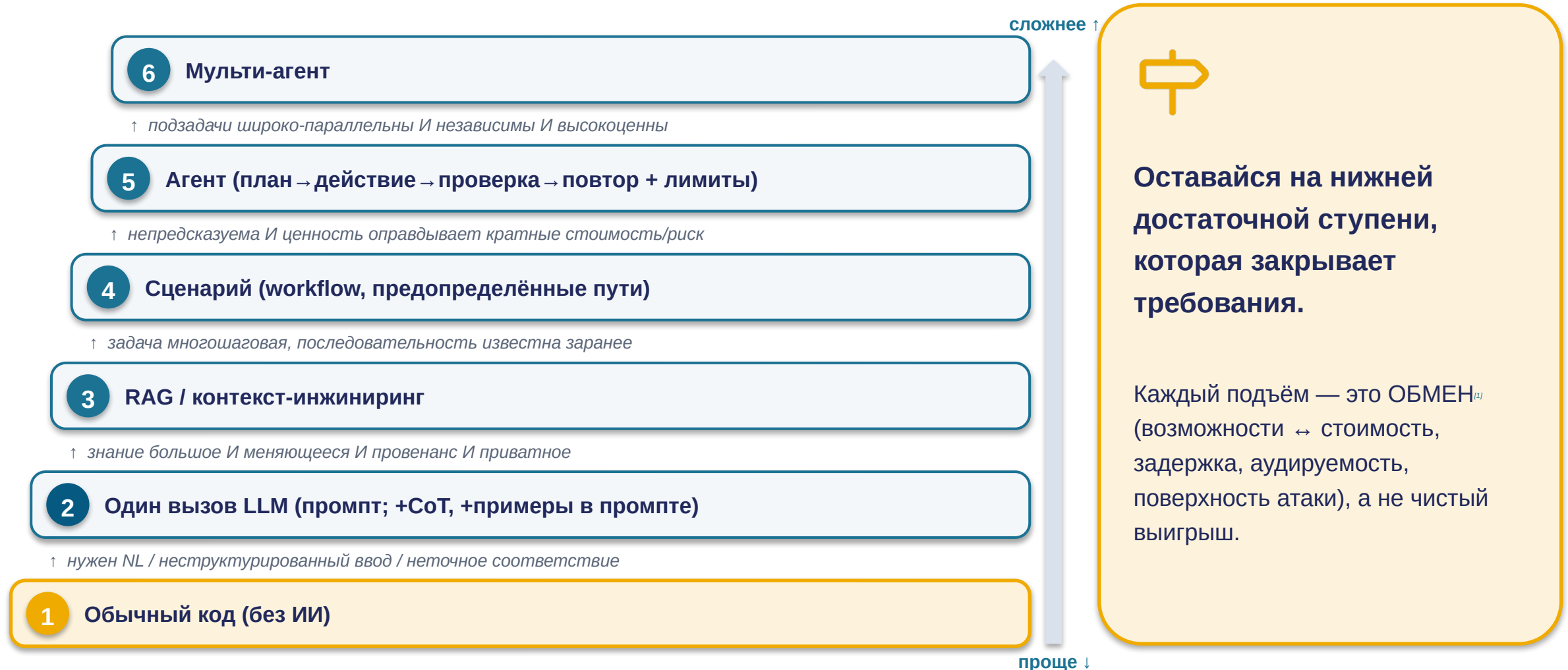
ЧТО ОСТАВИТЬ

Простейшая достаточная архитектура по умолчанию, а бремя доказательства — на том, кто хочет её усложнить. В AI-эпоху принцип не отменяется, а дорожает: лишняя ступень добавляет недетерминизм, стоимость токенов, задержку, поверхность атаки.

Мост: лестница архитектур (код → один вызов → RAG → workflow → агент → мульти-агент) — тот же принцип наименьшей мощности по AI-архитектурам: оставайся на нижней достаточной ступени, поднимайся только под требование.

Лестница архитектурной сложности.

Оставайся на нижней ступени; поднимайся только под требование задачи.



[1] Anthropic — Building Effective Agents (правило лестницы) · Нижняя ступень — «обычный код без ИИ»: лестница начинается с вопроса «а нужен ли ИИ вообще».

Стартовый комплект агента — и когда его усложнять.

Та же лестница «не усложняй без требования», применённая на уровень ниже — к оснастке одного агента, а не системы целиком.



По умолчанию — тонкий агент

- один файл-инструкция
- плоская память
- БЕЗ субагентов
- минимальный набор навыков
- минимальный MCP-доступ

Бремя доказательства — на усложнении, а не на простоте.



Память-бэкенд — когда:

история переросла контекст ИЛИ нужен поиск по фактам. Тот же критерий, что промпт → RAG.



Субагенты — когда:

подзадача требует отдельного окна ИЛИ изоляции недоверенной работы (наименьшие привилегии).



Больше MCP-доступа — когда:

конкретная задача требует конкретного инструмента — не «на всякий случай». Каждое подключение — граница доверия.



Тот же принцип, что лестница архитектур — применённый к оснастке одного агента: presence paradox показал, что даже файл-инструкция «как ритуал» не работает. Усложняй под конкретный проверяемый триггер.

Дальше — отрасли. Старт: разработка ПО, ось SDLC × AI.

Лекция 4 берёт тот же аппарат и раскладывает его по этапам жизненного цикла ПО. Каждый этап опирается на якорь из этой лекции:

Требования / дизайн

выбор архитектуры под шаг → лестница из 6 ступеней

Кодирование

агенты для кода через рамку экипировки; провалы как КЛАСС

Тестирование

граница доверия и наименьшие привилегии; данные вне ZDR

Эксплуатация

человек-валидатор проверяет результат, а не самообъяснение

Задание — Семинар 3: прогнать чек-лист выбора архитектуры на 3 кейсах (чат / агент / RAG / API).

Что делать: эта рамка — база для Лекций 4–17. На каждой отраслевой лекции прогоняйте тот же чек-лист выбора.



**жду ваши
вопросы**



Q&A

Спасибо